

■■ Amazon Web Services

Database Services

Complete Learning Guide

- Amazon DocumentDB
- Amazon DynamoDB
- Amazon Keyspaces
- Amazon MemoryDB for Redis
- Amazon Neptune
- Amazon RDS

From Simple Introduction to Real-World Architecture

Beginner-Friendly • In-Depth • Practical

■ Table of Contents

#	Service	Description
Chapter 1	Amazon DocumentDB	MongoDB-compatible document database
Chapter 2	Amazon DynamoDB	Serverless key-value & document NoSQL DB
Chapter 3	Amazon Keyspaces	Managed Apache Cassandra service
Chapter 4	Amazon MemoryDB for Redis	Durable in-memory database
Chapter 5	Amazon Neptune	Managed graph database service
Chapter 6	Amazon RDS	Managed relational database service

Each chapter covers: Introduction • Core Concepts • Architecture • Features • Use Cases • Integrations • Comparison

Amazon DocumentDB

MongoDB-Compatible Managed Document Database

1. Simple Introduction

Amazon DocumentDB is a fully managed, MongoDB-compatible document database service provided by AWS. It is designed to store, query, and index JSON-like data (called documents) at any scale.

Why Does DocumentDB Exist?

Traditional relational databases (like MySQL) store data in rigid rows and columns. But modern applications—social networks, content catalogs, user profiles—deal with flexible, nested data that changes shape frequently. DocumentDB was built to handle this flexible, schema-less data without the operational burden of managing your own MongoDB cluster.

■ Real-World Analogy

Think of DocumentDB like a digital filing cabinet where each drawer holds folders (collections), and each folder contains documents (like a user's profile card). Each card can have a different number of fields — one user might have 3 phone numbers, another might have none. Unlike Excel (relational DB), you don't need every row to match the same columns.

■ Business Problem Solved

- Applications need to store flexible, evolving data structures (no fixed schema)
- Teams using MongoDB on-premises want to migrate to a managed cloud service
- Need automatic backups, patching, replication — without a DBA team
- Storing product catalogs, user profiles, blog posts, IoT sensor data

2. Core Concepts

Key Terminology

Term	Definition	SQL Equivalent
Document	A JSON-like data record (BSON format internally)	Row / Record
Collection	A group of documents (like a table)	Table
Field	A key-value pair inside a document	Column
Index	Data structure for fast lookups	Index

Term	Definition	SQL Equivalent
Cluster	The DocumentDB server infrastructure (primary + replicas)	Database Server
Instance	A single database node in the cluster	Database Instance
Replica	A read-only copy of the primary node for HA and reads	Read Replica
Endpoint	The connection URL for your application	Connection String
Change Streams	Real-time stream of data change events from collections	Triggers / CDC

Document Structure Example

■ JSON Document — User Profile

```
{ "_id": "user_001",  
  "name": "Raj Patel",  
  "email": "raj@example.com",  
  "address": { "city": "Surat", "state": "Gujarat" },  
  "orders": [ { "id": "ORD-1", "amount": 1200 }, { "id": "ORD-2", "amount": 850 } ],  
  "premium_member": true  
}
```

■ Notice: 'orders' is an array of nested objects — impossible to represent cleanly in a single SQL row!

3. Architecture & Working

How DocumentDB Works Internally

DocumentDB separates compute (database instances) from storage. This is a distributed, fault-tolerant storage layer that automatically replicates your data 6 times across 3 Availability Zones (AZs).

Feature	Description	Benefit
6-Way Replication	6 copies across 3 AZs always	Extremely high durability (99.99%)
Up to 15 Read Replicas	Scale reads horizontally	Handle millions of read queries
Automated Backups	Daily backups, point-in-time recovery up to 35 days	Data recovery anytime
Encryption	At-rest (AWS KMS) and in-transit (TLS)	Security compliance
Change Streams	Real-time event stream of data changes	Build event-driven apps
Global Clusters	Replicate data across AWS Regions	Low-latency global reads
VPC Isolation	Runs inside your private VPC	Network-level security

5. Real-World Use Cases

Industry	Use Case	Why DocumentDB?
E-Commerce	Product catalog with varying attributes (size, color, warranty)	Each product has different fields
Social Media	User profiles, posts, comments, likes (nested data)	Flexible, nested JSON structure
Gaming	Player state, inventory, achievements	Fast writes, flexible schema
Healthcare	Patient records with varying fields per specialty	Schema-less medical records
Content Mgmt	Blog posts, articles, metadata, tags	Rich document storage
IoT	Sensor readings with varying data fields per device type	Flexible per-device schemas
Banking	Customer KYC documents with varied fields per customer type	Nested document support

6. AWS Service Integrations

AWS Service	How It Integrates	Use Case
AWS Lambda	Trigger functions on Change Streams	Real-time event processing
Amazon S3	Export snapshots to S3 for analytics	Data archival and analytics
AWS Glue	ETL from DocumentDB to data warehouse	Data pipeline
Amazon CloudWatch	Metrics, logs, alarms for monitoring	Operations & alerting
AWS Secrets Manager	Store DB credentials securely	Credential management
AWS IAM	Control access via IAM roles	Security & access control
Amazon VPC	Network isolation and private subnets	Network security
AWS KMS	Encrypt data at rest	Data encryption

7. Comparison with Similar Services

Feature	DocumentDB	DynamoDB	RDS MySQL
Data Model	Document (JSON)	Key-Value / Document	Relational (Tables)
Schema	Flexible	Flexible	Fixed
Query Language	MongoDB API	PartiQL / DynamoDB API	SQL
Scaling	Vertical + Read Replicas	Fully Serverless	Vertical + Read Replicas
Max Storage	128 TB	Unlimited	64 TB
Best For	MongoDB workloads	Massive scale, simple queries	Structured relational data
MongoDB Compat.	■ Yes	■ No	■ No

- **When to Choose DocumentDB**

 - You are already using MongoDB and want to migrate to AWS
 - Your data is JSON-like with varying structures across records
 - You need rich querying (nested fields, arrays, aggregations)
 - Avoid if you need ACID transactions across many tables — use RDS
 - Avoid if you need infinite serverless scale — use DynamoDB

■ Amazon DynamoDB

Serverless, Fully Managed NoSQL Database at Any Scale

1. Simple Introduction

Amazon DynamoDB is a fully managed, serverless, key-value and document NoSQL database that delivers single-digit millisecond performance at any scale. It requires zero server management — AWS handles everything including hardware provisioning, setup, configuration, replication, patching, and scaling.

■ Real-World Analogy

DynamoDB is like a massive, self-managing warehouse. You tell it what you want to store (put an item) and give it a unique label (primary key). When you need it, you hand over the label and get your item back instantly — no matter if the warehouse has 1,000 or 1 billion items. You never worry about adding shelves; the warehouse expands itself automatically.

■ Business Problem Solved

- Applications needing guaranteed millisecond response at millions of requests/sec
- No infrastructure to manage — pay per request or provisioned capacity
- Shopping carts, session stores, leaderboards, IoT telemetry
- Need global, multi-region data with automatic replication

2. Core Concepts

Fundamental Terminology

Term	Definition	Example
Table	Top-level container for items	'Orders' table
Item	A single record in a table (like a row)	One order record
Attribute	A field within an item (like a column)	'order_id', 'amount'
Primary Key	Unique identifier for each item	Partition Key alone, or PK + Sort Key
Partition Key	Hash key that determines which partition stores the item	'user_id'
Sort Key	Secondary key to sort items within a partition	'order_date'

Term	Definition	Example
GSI (Global Secondary Index)	Index with different PK+SK than the table — enables alternate query patterns	Query orders by status
LSI (Local Secondary Index)	Index with same PK but different SK — must be defined at table creation	Query orders by amount
Capacity Units	Unit of throughput: 1 RCU = 1 strong-consistent read of 4 KB	RCU = Read Capacity Unit
Streams	Ordered log of item-level changes (add/update/delete)	Event-driven triggers
TTL (Time to Live)	Auto-delete items after a specified expiry timestamp	Session cleanup
DAX	DynamoDB Accelerator — in-memory cache for microsecond reads	Cache layer

Primary Key Types

Key Type	Components	When to Use	Example
Simple (Partition Key only)	Partition Key	Items uniquely identified by one attribute	User table: user_id
Composite (Partition + Sort Key)	Partition Key + Sort Key	Multiple items per partition, query by range	Orders: user_id + order_date

Read Consistency Modes

Mode	Description	Cost	Use When
Eventually Consistent	May return slightly stale data (milliseconds behind)	1 RCU per 4KB	Read-heavy, stale data acceptable
Strongly Consistent	Always returns latest data	2 RCU per 4KB	Financial, inventory — must be current
Transactional	ACID transactions across multiple items/tables	4 RCU per 4KB	Cart checkout, financial operations

3. Architecture & Working

Internal Architecture

DynamoDB distributes data across multiple storage nodes using consistent hashing based on the Partition Key. Each item is stored in exactly one partition, and each partition is replicated 3 times across 3 AZs automatically.

Company / Industry	Use Case	Why DynamoDB?
Amazon (Retail)	Shopping cart — stores every user's cart	Millions of concurrent carts, ms latency
Gaming (Fortnite-style)	Player leaderboards, game state, inventory	Extreme write throughput, ms reads
Fintech (Stripe-style)	Idempotency keys, event log, transaction IDs	Guaranteed unique IDs, ACID transactions
Healthcare	Patient appointment tokens, session management	TTL auto-cleans expired sessions
IoT Platforms	Device telemetry — billions of readings/day	Scales to billions of writes automatically
Ride-sharing (Uber-style)	Driver location updates (thousands/second)	High write throughput, simple key-value
Startups	Backend for mobile apps (serverless APIs)	No DB admin, pay per use

6. AWS Service Integrations

AWS Service	Integration Method	Use Case
AWS Lambda	DynamoDB Streams → Lambda triggers	Serverless event processing
Amazon S3	Export table data to S3 (Parquet/JSON)	Analytics & data lake
Amazon Kinesis Data Streams	Stream DynamoDB changes to Kinesis	Real-time analytics pipelines
AWS AppSync	GraphQL APIs backed by DynamoDB	Mobile & web GraphQL backends
Amazon EMR	Process DynamoDB data with Spark	Big data analytics
Amazon CloudWatch	Monitor RCU/WCU usage, throttles	Performance monitoring
AWS IAM	Fine-grained access control per table	Security
DAX	In-memory cache in front of DynamoDB	Microsecond reads

7. Comparison

Feature	DynamoDB	DocumentDB	RDS	ElastiCache
Type	Key-Value/Document	Document	Relational	In-Memory Cache
Schema	Flexible	Flexible	Fixed	N/A
Scale	Infinite, auto	Up to 128TB	Up to 64TB	RAM-limited
Latency	<10 ms	<10 ms	<100 ms	<1 ms
SQL Support	PartiQL (limited)	MongoDB API	Full SQL	No
Serverless	■ Native	■ No	■ Aurora Serverless	■ No

Feature	DynamoDB	DocumentDB	RDS	ElastiCache
ACID Transactions	■ Yes	■ Yes	■ Yes	■ No
Best For	Massive scale apps	MongoDB migration	Relational workloads	Ultra-fast caching

Amazon Keyspaces

Managed Apache Cassandra-Compatible Database Service

1. Simple Introduction

Amazon Keyspaces (for Apache Cassandra) is a scalable, highly available, and fully managed Cassandra-compatible database service. It enables you to run your Cassandra workloads on AWS without managing Cassandra clusters yourself.

Apache Cassandra is a popular open-source wide-column NoSQL database used by companies like Netflix, Instagram, and Discord for storing massive amounts of time-series and write-heavy data.

■ Real-World Analogy

Think of Keyspaces like a giant spreadsheet where rows can be incredibly wide — each row can have thousands of columns. It's optimized for writing millions of rows per second (like recording every user's every action on a social media platform). The 'keyspace' is like a schema, and each 'table' stores rows that can have different numbers of columns.

■ Business Problem Solved

- Run Cassandra workloads without managing servers, clusters, or nodes
- Time-series data: IoT sensor readings, log data, financial tick data
- Write-intensive workloads needing extremely high throughput
- Migrate on-premises Cassandra to AWS with CQL compatibility

2. Core Concepts

Term	Definition	SQL Equivalent
Keyspace	Top-level namespace (like a schema or database)	Database / Schema
Table	A collection of rows with a defined schema	Table
Row	A single record in a table	Row
Column	A field in a row	Column
Partition Key	Determines which node/partition stores the row	Primary Key (partial)
Clustering Column	Sorts rows within a partition	Sort Key / Index

Feature	Description	Benefit
TTL Support	Automatically expire rows	Time-series data management
IAM Integration	Fine-grained access control	Security
Audit Logging	Via AWS CloudTrail	Compliance

5. Real-World Use Cases

Industry	Use Case	Why Keyspaces?
IoT Platforms	Store billions of sensor readings per day (time-series)	Optimized for time-series, high writes
Fintech	Financial tick data, trade history, audit logs	Ordered data within partition, fast writes
Social Media	Activity feeds, messaging history, notifications	Wide-column, massive write throughput
Telecom	Call Detail Records (CDR), network event logs	Billions of records, fast sequential writes
Gaming	Player event logs, match history	High-volume event capture
Netflix-style	Viewing history per user across all devices	Wide-column, partition by user_id

6. AWS Service Integrations

AWS Service	Integration	Use Case
AWS Glue	ETL pipelines from Keyspaces	Data transformation & analytics
Amazon S3	Export table data to S3	Data lake & archival
Amazon CloudWatch	Metrics and alarms	Monitoring & alerting
AWS IAM	Table-level access control	Security
AWS KMS	Encryption key management	Data security
AWS CloudTrail	API audit logs	Compliance & audit

7. Keyspaces vs DynamoDB vs Cassandra

Feature	Keyspaces	DynamoDB	Self-Managed Cassandra
Type	Wide-column (Cassandra)	Key-Value/Document	Wide-column
Query Lang	CQL (Cassandra SQL)	PartiQL / DynamoDB API	CQL
Managed	■ Fully	■ Fully	■ You manage
Cassandra Compat	■ Yes	■ No	■ Yes (native)

Feature	Keyspaces	DynamoDB	Self-Managed Cassandra
Serverless	■ Yes	■ Yes	■ No
ACID Transactions	■ Limited	■ Yes	■ Limited
Best For	Cassandra migrations, time-series	General NoSQL at scale	Full Cassandra control

Amazon MemoryDB for Redis

Durable, In-Memory Database with Redis Compatibility

1. Simple Introduction

Amazon MemoryDB for Redis is a fully managed, Redis-compatible in-memory database service designed for workloads that require ultra-fast performance AND data durability. Unlike ElastiCache (which is a cache that can lose data), MemoryDB stores all data durably in a distributed transaction log.

■ Real-World Analogy

Traditional Redis (or ElastiCache) is like RAM in your computer — blazing fast but data is lost on restart. MemoryDB is like RAM + a hard drive fused together: you get the speed of RAM with the safety of disk. If the server restarts, your data is still there — instantly available at the same microsecond speed.

■ Business Problem Solved

- Need Redis speed but CANNOT afford to lose data (financial transactions, session state)
- Want to use Redis as a primary database, not just a cache
- Need durable leaderboards, real-time counters, queues that survive restarts
- Microservices needing ultra-low-latency data that must be persistent

2. Core Concepts

Term	Definition
In-Memory Database	All data stored in RAM for microsecond access speeds
Redis Compatibility	Supports all Redis 6.x data structures and commands
Durable Transaction Log	Multi-AZ log that persists every write before acknowledging to client
Cluster	Group of nodes (primary + replicas) that store and serve data
Shard	A horizontal partition of data across the cluster
Primary Node	The read-write node in each shard
Replica Node	Read-only copies of the primary for HA and read scaling
Multi-AZ	Replicas and transaction log span multiple Availability Zones
Failover	Automatic promotion of a replica to primary if primary fails

Term	Definition
Snapshot	Point-in-time backup of the entire dataset to S3
Redis Data Structures	Strings, Hashes, Lists, Sets, Sorted Sets, Streams, Bitmaps

MemoryDB vs ElastiCache for Redis

Dimension	MemoryDB for Redis	ElastiCache for Redis
Primary Role	PRIMARY DATABASE (durable)	CACHE (ephemeral)
Data Durability	■ Yes — Multi-AZ transaction log	■ No — data may be lost on failure
Data Loss Risk	None (full ACID durability)	Cache miss possible on failure
Use As Primary DB	■ Yes	■ Not recommended
Performance	Microseconds (RAM-speed)	Microseconds (RAM-speed)
Cost	Higher (durability overhead)	Lower
Redis API Compat	■ Yes	■ Yes
Best For	Session store, leaderboards, queues as primary	Caching DB query results

3. Architecture & Working

■ MemoryDB Architecture

YOUR APP → MemoryDB Cluster Endpoint

© 2015 Pearson Education, Inc. or its affiliate(s). All rights reserved. Pearson Education, Inc., 501 Boylston Street, Boston, MA 02116

■ SHARD 1 ■ SHARD 2 ■

■ Primary (AZ-1) ← ■ ■ ■ ■ Primary (AZ-2) ■

■ Replica (AZ-2) ■ ■ Replica (AZ-3) ■

■ Replica (AZ-3) ■ ■ Replica (AZ-1) ■

Page 10 of 10

© 2015 Pearson Education, Inc. or its affiliate(s). All rights reserved. Pearson Education, Inc., 501 Boylston Street, Boston, MA 02116

MULTI-AZ TRANSACTION LOG

■ ■ Every write persisted BEFORE ■ ■

■ ■ acknowledging to client ■ ■

© 2015 Pearson Education, Inc. or its affiliate(s). All rights reserved. Pearson Education, Inc., 501 Boylston Street, Boston, MA 02116

© 2015 Pearson Education, Inc. or its affiliate(s). All rights reserved. Pearson Education, Inc., publishing as Pearson Benjamin Cummings, 101 Philip Drive, Assinippi Park, New York, NY 10964-2133. All rights reserved. Printed in the United States of America. This publication is protected by copyright. Permission to reproduce copies may be obtained from the Copyright Clearance Center, Inc., 222 Rosewood Drive, Danvers, MA 01923. For more information, contact CCR, www.copyright.com.

Result: Microsecond reads + Zero data loss guarantee

4. Key Features

Feature	Description	Benefit
Microsecond Latency	Data served from RAM	Sub-millisecond response times
Durable Transaction Log	Multi-AZ write-ahead log	Zero data loss even on failure
Redis API Compatibility	Full Redis 6.x command support	No app code changes from Redis
Multi-AZ Replication	Data replicated across AZs automatically	High availability
Automatic Failover	Failed primary replaced by replica in seconds	Self-healing
Online Scaling	Add shards/replicas without downtime	Scale without maintenance windows
Encryption	TLS in-transit, KMS at-rest	Security compliance
Snapshots	Point-in-time backups to S3	Disaster recovery
VPC Support	Runs in your private VPC	Network isolation

5. Real-World Use Cases

Use Case	Description	Key Redis Structure Used
Session Store	Store user login sessions durably — survive server restarts	Hash (user session data)
Gaming Leaderboard	Real-time ranked leaderboard with persistent scores	Sorted Set (ZADD/ZRANGE)
Real-Time Counters	Page views, likes, API rate limiting — never lose counts	String (INCR)
Job Queue	Task queue for background jobs — durable even on failure	List (LPUSH/BRPOP)
Pub/Sub Messaging	Microservices event streaming (durable)	Streams (XADD/XREAD)
Inventory Management	Real-time stock count updates — must not lose data	Hash + Atomic Operations
ML Feature Store	Low-latency feature serving for ML models	Hash / String

6. AWS Service Integrations

AWS Service	Integration	Use Case
Amazon EC2	App servers connect to MemoryDB cluster	Primary app database
AWS Lambda	Serverless functions read/write MemoryDB	Serverless backends
Amazon ECS/EKS	Containerized apps connect to MemoryDB	Microservices data store
Amazon CloudWatch	Monitor memory, CPU, connections, commands	Performance monitoring

AWS Service	Integration	Use Case
AWS Secrets Manager	Store Redis credentials securely	Credential management
Amazon VPC	Network isolation	Security

Amazon Neptune

Fully Managed Graph Database Service

1. Simple Introduction

Amazon Neptune is a fully managed graph database service that makes it easy to build and run applications that work with highly connected datasets. Neptune supports two popular graph models: Property Graph (using Gremlin or openCypher) and RDF (Resource Description Framework, using SPARQL).

A graph database stores data as nodes (entities) and edges (relationships), which is fundamentally different from tables (relational) or documents (document DB). It is designed to traverse complex relationships efficiently — something impossible at scale with SQL JOINS.

■ Real-World Analogy

Think of Neptune like LinkedIn's database. Every user is a node. Every 'connection' between users is an edge. If LinkedIn asked 'show me all friends-of-friends of Raj who work at AWS' — a SQL database would need 3+ JOINS across millions of rows. Neptune traverses the graph directly, answering in milliseconds even with billions of relationships.

■ Business Problem Solved

- Fraud detection: find suspicious transaction patterns across thousands of connected accounts
- Social network: friends, followers, connections, recommendations
- Knowledge graph: relationships between concepts, products, people
- Network/IT infrastructure topology mapping
- Drug interaction graphs in life sciences

2. Core Concepts

Graph Data Model Terminology

Term	Definition	Example
Node (Vertex)	An entity in the graph	Person, Product, Account
Edge	A relationship between two nodes	'KNOWS', 'BOUGHT', 'TRANSFERS_TO'
Property	Attributes on nodes or edges	name='Raj', amount=500

Term	Definition	Example
Label	Category/type tag on a node or edge	'Person', 'Transaction'
Graph Traversal	Walking through nodes and edges to answer queries	'Find friends of friends'
Property Graph	Graph model where nodes/edges have properties	LinkedIn, fraud graphs
RDF (Resource Description Framework)	Triple-based graph model (subject-predicate-object)	Knowledge graphs, semantic web
Gremlin	Graph traversal language for Property Graphs (Apache TinkerPop)	Traversal queries
openCypher	SQL-like graph query language (used in Neo4j)	Declarative graph queries
SPARQL	Query language for RDF graphs	Semantic web queries
Cluster	Neptune primary + up to 15 read replicas	High availability setup

Supported Query Languages

Language	Graph Model	Use Case	Style
Gremlin	Property Graph	Social networks, fraud detection	Imperative (step-by-step traversal)
openCypher	Property Graph	Same as Gremlin, simpler syntax	Declarative (SQL-like)
SPARQL	RDF Graph	Knowledge graphs, semantic data	Triple-pattern matching

3. Architecture & Working

■ ■ Neptune Architecture

YOUR APP → Neptune Cluster Endpoint



■ PRIMARY DB INSTANCE (Reads + Writes) ■



■ Replicated via shared storage



■ REPLICA 1 ■ REPLICA 2 (up to 15) ■

■ (Read-only) ■ (Read-only) ■



■ DISTRIBUTED STORAGE LAYER ■

■ 6 copies across 3 AZs ■

■ SSD-backed, up to 128 TB ■



4. Key Features

Feature	Description	Benefit
Multi-Model Support	Property Graph (Gremlin/openCypher) + RDF (SPARQL)	Flexibility for different graph types
Managed Service	AWS handles patching, backups, failover	No graph DB expertise needed
High Performance	Optimized for graph traversals, billions of relationships	Millisecond query times
Up to 15 Replicas	Scale reads across 15 read replicas	High-throughput graph reads
Serverless (Neptune Serverless)	Auto-scales compute capacity	Variable/unpredictable workloads
Neptune Analytics	In-memory graph analytics for batch/OLAP graph queries	Graph analytics at scale
Streams	Change data capture for graph changes	Event-driven graph processing
ML Integration (Neptune ML)	Graph neural networks using SageMaker	ML on graph data
Global Database	Cross-region read replicas	Low-latency global graph reads
6-Way Storage Replication	Always 3 AZs, 6 copies	Durability and HA
Encryption	KMS at-rest, TLS in-transit	Security compliance

5. Real-World Use Cases

Industry	Use Case	Query Example
Financial / Banking	Fraud Detection: find ring of accounts passing money in a circle	Find transaction cycles of length 3-5 hops
Social Networks	Friend recommendations, mutual connections, influence analysis	Friends-of-friends within 2 hops
E-Commerce	Product recommendation: users who bought X also bought Y	Graph-based collaborative filtering
Life Sciences	Drug interaction graphs, protein-protein networks	Find all drugs interacting with enzyme X
Cybersecurity	Attack path analysis, network topology mapping	Shortest path between compromised node and sensitive data
Knowledge Graph	Google Knowledge Graph-style entity relationships	What is Amazon connected to?
HR / Org Chart	Reporting hierarchy, skills graph	Find all engineers reporting to VP of Eng
IT Operations	Network infrastructure dependency graph	Impact analysis of node failure

6. AWS Service Integrations

AWS Service	Integration	Use Case
Amazon SageMaker	Neptune ML — train graph neural networks on Neptune data	ML on graph data
AWS Lambda	Process Neptune Streams events	Event-driven graph updates
Amazon S3	Load graph data from S3 (bulk loader)	Bulk data ingestion
Amazon OpenSearch	Full-text search + graph traversal combined	Hybrid search + graph queries
Amazon CloudWatch	Metrics, logs, alarms	Monitoring
AWS IAM	Access control via IAM roles	Security
Amazon VPC	Network isolation	Security

7. Neptune vs Other Databases

Feature	Neptune	RDS (SQL)	DynamoDB	DocumentDB
Data Model	Graph (nodes+edges)	Tables (rows+cols)	Key-Value/Doc	Document (JSON)
Relationship Handling	Native (traversal)	JOINS (expensive)	Manual (complex)	Embedded docs
Query Language	Gremlin/openCypher/SPARQL	SQL	PartiQL	MongoDB API

Feature	Neptune	RDS (SQL)	DynamoDB	DocumentDB
Managed	■ Yes	■ Yes	■ Yes	■ Yes
Best For	Connected data, fraud, social	Structured business data	High-scale NoSQL	JSON document storage
Multi-hop Queries	■ Fast (ms)	■ Slow (many JOINS)	■ Not native	■ Not native

■ Amazon RDS

Managed Relational Database Service — MySQL, PostgreSQL, Oracle, SQL Server & More

1. Simple Introduction

Amazon RDS (Relational Database Service) is a fully managed relational database service that makes it easy to set up, operate, and scale relational databases in the cloud. RDS supports 6 popular database engines: Amazon Aurora, MySQL, MariaDB, PostgreSQL, Oracle, and SQL Server.

RDS handles time-consuming database administration tasks such as hardware provisioning, database setup, patching, and backups — so you can focus on building applications.

■ ■ Real-World Analogy

Running your own database on EC2 is like building and running your own restaurant kitchen — you buy equipment, hire chefs, maintain appliances, and clean up after service. Amazon RDS is like hiring a catering company: you tell them what food (data) you need, they provide and maintain the kitchen, and you just focus on your business (application). If a stove breaks, they fix it automatically.

■ Business Problem Solved

- Use standard SQL databases without managing servers, OS, or DB software
- Automatic backups, patching, multi-AZ failover — zero DBA work
- Lift-and-shift: migrate on-premises MySQL/PostgreSQL to cloud with no code changes
- ACID-compliant transactional databases for ERP, CRM, finance, e-commerce

2. Core Concepts

Supported Database Engines

Engine	Type	Best For	Key Strength
Amazon Aurora (MySQL)	AWS-optimized MySQL	High-performance MySQL workloads	5x faster than MySQL, serverless option
Amazon Aurora (PostgreSQL)	AWS-optimized PostgreSQL	High-performance PG workloads	3x faster than PostgreSQL, global
MySQL	Open-source relational	Web apps, CMS, e-commerce	World's most popular open-source DB

Engine	Type	Best For	Key Strength
PostgreSQL	Advanced open-source relational	Complex queries, analytics, GIS	JSON support, advanced features
MariaDB	MySQL fork	MySQL alternative	Community-driven, open-source
Oracle	Enterprise commercial	Legacy enterprise apps	Enterprise features, Oracle compatibility
SQL Server	Microsoft commercial	Windows/.NET ecosystems	T-SQL, SSRS, SSIS integration

Key Terminology

Term	Definition
DB Instance	A single database server running in the cloud (primary or replica)
DB Instance Class	The compute (CPU/RAM) size — e.g., db.t3.micro, db.r6g.4xlarge
Multi-AZ Deployment	Primary in one AZ, standby in another — automatic failover in 60-120 sec
Read Replica	Read-only copy of primary for scaling read traffic (up to 5 for MySQL/PG)
Automated Backups	Daily snapshots + transaction logs, retained 1-35 days
DB Snapshot	Manual point-in-time backup stored in S3, kept until you delete it
Parameter Group	Configuration settings for the DB engine (like my.cnf)
Option Group	Additional features for the DB engine (e.g., Oracle TDE)
Subnet Group	Set of subnets across AZs where RDS can be deployed
Security Group	Virtual firewall controlling network access to the DB instance
Storage Types	gp2 (general SSD), gp3 (tunable SSD), io1 (high IOPS SSD), magnetic
PITR (Point-in-Time Recovery)	Restore DB to any second within the backup retention period
Maintenance Window	Weekly window for applying patches and minor upgrades
Enhanced Monitoring	OS-level metrics at 1-second granularity
Performance Insights	Query-level performance analytics — find slow queries

3. Architecture & Working

Standard RDS Architecture (Multi-AZ)

■ ■ RDS Multi-AZ Architecture

YOUR APP (EC2 / Lambda / ECS)

↓ SQL Queries (via RDS Endpoint)



■ PRIMARY DB INSTANCE (AZ-1) ■

■ Handles ALL reads and writes ■

■ Storage: EBS (gp3/io1) ■



■ Synchronous Replication



■ STANDBY INSTANCE (AZ-2) ■

■ Exact copy, NOT serving queries ■

■ Auto-promoted to primary on failure (60s) ■



■ READ REPLICAS (Optional, up to 5) ■

■ Asynchronous replication from primary ■

■ Serve SELECT queries → offload primary ■



Replication Types

Type	Method	Lag	Purpose	Failover?
Multi-AZ Standby	Synchronous (every write mirrored)	0 (sync)	High Availability	■ Auto (60-120s)
Read Replica	Asynchronous (slight lag)	Seconds	Read Scaling	■ Manual promotion
Cross-Region Replica	Asynchronous	Minutes	DR & global reads	■ Manual promotion

4. Amazon Aurora — The Premium RDS Engine

Amazon Aurora is AWS's own cloud-native relational database compatible with MySQL and PostgreSQL, but rebuilt for the cloud. It delivers dramatically better performance, availability, and scalability than standard MySQL or PostgreSQL.

Aurora Feature	Description	vs Standard MySQL/PG
Storage Auto-scaling	Grows 10 GB at a time, up to 128 TB automatically	MySQL: manual resize needed
6-Way Replication	6 copies across 3 AZs in the storage layer	MySQL Multi-AZ: 2 copies
Up to 15 Read Replicas	Low-lag (10ms typically) read replicas	MySQL: up to 5, higher lag

Aurora Feature	Description	vs Standard MySQL/PG
Aurora Serverless v2	Auto-scales compute in seconds, pay per second	MySQL: fixed instance sizes
Global Database	Replicate across 5 AWS Regions, <1 sec lag	MySQL: cross-region replication lag is higher
Backtrack	Rewind DB to any point in last 72 hours without restore	MySQL: restore from backup only
Aurora ML	Query ML models (SageMaker, Comprehend) via SQL	Not available in MySQL/PG
Fault Tolerant Storage	Handles loss of 2 copies, still reads; loss of 3, still writes	MySQL: AZ failure = potential downtime

5. Key RDS Features

Feature	Description	Business Value
Automated Backups	Daily full backup + continuous transaction log backup	Restore to any second
Multi-AZ HA	Synchronous standby in another AZ	<2 min failover, zero data loss
Read Replicas	Up to 5 async read copies (15 for Aurora)	Scale reads 5-15x
Storage Auto Scaling	Auto-grow storage when threshold crossed	Avoid out-of-space issues
Performance Insights	Visual query performance dashboard	Identify top slow queries instantly
Enhanced Monitoring	60+ OS metrics at 1-second granularity	Deep system-level visibility
Encryption	KMS at-rest, SSL/TLS in-transit, always-on for Aurora	Security compliance
IAM Authentication	Use IAM tokens instead of passwords	Passwordless DB access
Parameter Groups	Tune DB engine settings without restart (dynamic params)	DB configuration management
Maintenance Window	Scheduled weekly window for patches	Controlled update schedule
Blue/Green Deployments	Create staging copy, test upgrades, then switchover in seconds	Zero-downtime upgrades
Proxy (RDS Proxy)	Connection pooling for serverless apps (Lambda)	Reduces DB connection overload

6. Real-World Use Cases

Industry	Engine Choice	Use Case
E-Commerce (Flipkart)	Aurora MySQL	Product catalog, orders, payments — high traffic, structured data
Banking / Fintech	PostgreSQL / Oracle	ACID transactions, ledgers, loan management, regulatory compliance

Industry	Engine Choice	Use Case
Healthcare	PostgreSQL	Patient records, appointment scheduling, HIPAA compliance
SaaS Companies	Aurora PostgreSQL Serverless	Multi-tenant apps with variable traffic — pay only when active
ERP / CRM Systems	Oracle / SQL Server	SAP, Dynamics 365 — lift-and-shift from on-premises
Gaming	Aurora MySQL	User accounts, game progress, in-app purchases
Logistics	MySQL / PostgreSQL	Shipment tracking, warehouse inventory, route planning
Content Platforms	MySQL	CMS backends (WordPress), user auth, content metadata

7. AWS Service Integrations

AWS Service	Integration	Use Case
AWS Lambda	RDS Proxy → Lambda connections pooled	Serverless apps with RDS backend
Amazon S3	Import/export data to/from S3	Data migration, batch loading
AWS DMS (Database Migration Service)	Migrate on-prem DB to RDS online	Zero-downtime migration
AWS Secrets Manager	Rotate DB passwords automatically	Credential security
Amazon CloudWatch	Instance metrics, slow query logs	Monitoring & alerting
Performance Insights	Query-level analysis dashboard	Database performance tuning
AWS Glue	ETL from RDS to data warehouse / S3	Analytics pipeline
Amazon Redshift	RDS → Redshift for OLAP analytics	Analytics on operational data
AWS IAM	IAM role-based DB authentication	Passwordless access
AWS KMS	Manage encryption keys for RDS	Data encryption
Amazon ElastiCache	Cache frequent SQL results in front of RDS	Reduce DB load
Amazon SNS	Event notifications (failover, backup complete)	Operational alerts

8. Comparison — All AWS Database Services

Service	Type	Schema	Scale	Latency	Best For
RDS	Relational	Fixed	Vertical+Replicas	ms	Structured SQL workloads
Aurora	Relational (AWS)	Fixed	Vertical+15 Replicas+Serverless	ms	High-perf SQL, cloud-native
DynamoDB	NoSQL Key-Value	Flexible	Infinite auto	<10 ms	Massive scale NoSQL

Service	Type	Schema	Scale	Latency	Best For
DocumentDB	Document (JSON)	Flexible	Up to 128 TB	ms	MongoDB workloads
Keyspaces	Wide-Column	Semi-fixed	Auto	ms	Time-series, Cassandra migration
MemoryDB	In-Memory (Redis)	Schema-less	RAM-limited	Microseconds	Durable Redis primary DB
Neptune	Graph	Graph model	128 TB	ms	Relationships, social, fraud
Redshift	Data Warehouse	Fixed	Petabyte scale	Seconds	Analytics, BI, OLAP
ElastiCache	Cache (Redis/Memcached)	Schema-less	RAM-limited	Microseconds	Caching, NOT primary DB

Decision Guide: Which Database to Choose?

■ Quick Selection Guide

- Structured data + SQL queries → Amazon RDS / Aurora
- MySQL/PostgreSQL + max performance → Amazon Aurora
- Huge scale + simple key-value lookups → Amazon DynamoDB
- JSON documents + MongoDB API → Amazon DocumentDB
- Time-series + Cassandra migration → Amazon Keyspaces
- Redis speed + MUST NOT lose data → Amazon MemoryDB
- Highly connected data + relationships → Amazon Neptune
- Analytics + BI + large dataset queries → Amazon Redshift
- Caching SQL results for performance → Amazon ElastiCache